

1. În arhitectura MVC, ce rol are componenta Controller?

Controller-ul generează interfața grafică și formatează

☐ datele în elemente HTML.

Controller-ul asigură doar verificarea datelor introduse în formulare, fără alte

☐ funcționalități.

Controller-ul gestionează cererile HTTP, transmite datele către Model și redirecționează

☐ rezultatele către View.

Controller-ul se ocupă cu stocarea datelor aplicației și menținerea conexiunii cu baza

☐ de date.

time remaining: 44 seconds

2. Ce reprezintă simbolul
? dintr-o rută definită ca:
{controller=Home}/{action=Index}/{

Specifică faptul că parametrul
id este opțional și poate lipsi
☐ din adresă fără a genera erori.

Definește un parametru
rezervat, folosit doar de
framework-ul ASP.NET pentru
☐ testare internă.

Marchează un parametru care
va fi întotdeauna ignorat de
☐ sistemul de rutare.

Indică faptul că parametrul id
este obligatoriu și trebuie
☐ transmis în URL.

SUBMIT

flexiquiz.com



3. Ce se întâmplă dacă două rute au același număr de parametri și sunt definite în ordine diferită în fișierul Program.cs?

Framework-ul ASP.NET Core va genera o eroare la pornirea aplicației din cauza conflictului
☐ de rute.

Ordinea nu contează, deoarece rutele sunt gestionate intern de
☐ motorul de compilare.

Sistemul va combina automat ambele rute într-o singură rută
☐ pentru a evita ambiguitățile.

Prima rută care apare în fișier va fi evaluată prima, iar dacă se potrivește, celelalte nu vor mai
☐ fi verificate.

4. Ce rezultat returnează aplicația pentru ruta definită astfel, dacă se accesează /Verify/150?

```
app.MapControllerRoute(  
    name: "HomePage",  
    pattern:  
    "Verify/{id:range(10,100)}",  
    defaults: new {  
        controller = "Home",  
        action = "Verify" });
```

Va returna un mesaj “Verify 150”, deoarece parametrul este
☐ numeric și trece de validare.

Va returna un mesaj care confirmă valoarea primită, deoarece orice parametru este
☐ acceptat implicit.

Va redirecționa automat către ruta default, pentru că regula
☐ de interval nu este aplicabilă.

Va returna un cod HTTP 404 (Not Found), deoarece valoarea 150 depășește limita maximă
☐ admisă de 100.

SUBMIT

time remaining: 41 seconds

5. Care este scopul clasei
WebApplication în cadrul
arhitecturii ASP.NET Core
9?

Este un model de date care gestionează entitățile și legăturile dintre tabelele bazei de date.

Este clasa care oferă o interfață unificată pentru configurarea serviciilor, rutelor și componentelor middleware ale aplicației.

Este clasa utilizată exclusiv pentru autentificare, gestionând utilizatorii și sesiunile acestora

Este o clasă auxiliară care permite doar configurarea fișierelor statice și a directorului wwwroot.

SUBMIT

time remaining: 58 seconds

6. De ce este necesar ca metodele unui Controller să fie declarate publice într-o aplicație ASP.NET Core MVC?

Deoarece numai metodele publice pot fi expuse către mecanismul de routing al framework-ului, permițând astfel ca acestea să fie mapate la request-urile HTTP primite din browser și executate ca acțiuni accesibile utilizatorilor finali.

Deoarece metodele publice sunt automat înregistrate în containerul de servicii al aplicației, facilitând gestionarea lor ca servicii reutilizabile și permițând injecția lor în alte

☐ componente MVC.

Deoarece o metodă publică poate fi accesată oricând de alte componente interne ale aplicației, inclusiv de alte controllere, facilitând astfel reutilizarea logicii și modularizarea funcționalităților

☐ din proiect.

Deoarece vizibilitatea publică permite ca metodele controller-ului să fie accesibile direct din View-uri, oferind posibilitatea de a transfera controlul execuției către acestea și de a utiliza rezultatele logice în generarea

☐ interfeței

7. Ce reprezintă
conceptul de
Dependency Injection în
cadrul framework-ului
ASP.NET Core și care este
rolul său în dezvoltarea
aplicațiilor web
moderne?

Dependency Injection este o
tehnică integrată în framework
care simplifică procesul de
creare și configurare a unei
baze de date, gestionând
automat conexiunile și relațiile
dintre tabele în timpul

☐ inițializării aplicației.



Dependency Injection este un design pattern fundamental care permite separarea clară a responsabilităților, facilitând scrierea unui cod scalabil, modular și ușor de întreținut, prin injectarea automată a dependențelor necesare

☐ componentelor aplicației.

Dependency Injection reprezintă un mecanism folosit în special pentru configurarea sistemului de rutare, permițând definirea flexibilă a regulilor de mapare a URL-urilor către

☐ controllere și acțiuni.

Dependency Injection este un instrument oferit de runtime pentru optimizarea performanței aplicației, reducând numărul de request-uri HTTP și îmbunătățind modul în care sunt servite

☐ fișierele statice din proiect.

8. Ce sunt acțiunile (Actions) în cadrul unui Controller și cum sunt ele utilizate în procesarea request-urilor HTTP?

Acțiunile sunt metode private definite în interiorul unui Controller, fiind invocate intern în timpul procesării unui request, fără a fi expuse în mod direct mecanismului de rutare al aplicației.

☐ al aplicației.

Acțiunile sunt metode publice declarate într-un Controller, care pot fi apelate de framework atunci când un request HTTP se potrivește unei rute, reprezentând punctul principal de execuție pentru

☐ logica specifică cererii.

Acțiunile sunt metode statice care gestionează exclusiv operațiunile de validare, fiind accesibile doar în interiorul zonei de configurare și nefiind implicate în rularea cererilor

☐ HTTP.

Acțiunile sunt rute configurate în fișierul Program.cs, responsabilitatea lor fiind doar să stabilească maparea URL-urilor către controllerele aplicației, fără a include logica

☐ internă.

9. Care este rolul atributului [HttpPost] aplicat unei metode dintr-un Controller în ASP.NET Core MVC?

Atributul [HttpPost] marchează metoda ca fiind responsabilă doar cu preluarea și afișarea datelor, fiind apelată exclusiv atunci când serverul trebuie să returneze informații în urma

☐ unei cereri de tip GET.

Atributul [HttpPost] indică faptul că metoda respectivă din Controller va gestiona cereri HTTP de tip POST, fiind folosită în special pentru scenarii în care datele sunt trimise de la client la server, cum ar fi formularele de creare sau

☐ modificare.

Atributul [HttpPost] este utilizat pentru a realiza redirectionarea automată a utilizatorilor către o altă pagină după încheierea executării acțiunii, fără a lua în considerare tipul cererii HTTP

☐ inițiale.

Atributul [HttpPost] este utilizat pentru a specifica faptul că metoda Controller-ului poate fi accesată doar din codul serverului și nu poate fi apelată printr-o rută directă din

☐ browser.



10. Care dintre următoarele afirmații descrie corect funcționalitatea Model Binding-ului în ASP.NET MVC Core?

Model Binding-ul gestionează conexiunile la baza de date și interogările SQL necesare pentru extragerea datelor utile
☐ răspunsurilor Controller-ului;

Model Binding-ul controlează fluxul de navigație într-o aplicație ASP.NET, determinând ce pagină sau View trebuie să fie afișată în urma unei acțiuni
☐ din Controller:



Model Binding-ul este responsabil pentru configurarea și administrarea sesiunilor utilizatorilor în aplicații, menținând starea între

☐ diferite cereri HTTP;

Model Binding-ul facilitează automat maparea valorilor din cererile HTTP la proprietățile unui Model, permițând preluarea eficientă a datelor din formularele trimise prin

☐ View;

11. Ce rol joacă un DbSet într-un DbContext, conform descrierii următoare?

```
public class
SchoolContext :
    DbContext
    {
public DbSet<Student>
    Students { get; set; }
public DbSet<Teacher>
    Teachers { get; set; }
    }
```



Proprietățile DbSet sunt utilizate pentru a defini relațiile

☐ dintre entități (1:1; 1:m; m:m);

Proprietățile DbSet sunt utilizate pentru a configura baza de date și nu sunt implicate în operațiunile de interogare sau actualizare a

☐ datelor;

DbSet în DbContext este folosit pentru a defini și executa reguli de business complexe, integrând logica de business direct la nivelul accesului la

☐ date;

DbSet într-un DbContext reprezintă o colecție de entități de un anumit tip, în acest caz, Student și Teacher, care poate fi utilizată pentru a interoga și a salva instanțe ale acestor

☐ entități în baza de date;

12. Completați codul astfel încât să afișați un mesaj temporar de succes după ce un produs este adăugat în baza de date. Mesajul trebuie să fie afișat pe pagina următoare, iar apoi să dispară din pagină după afișare.

```
public IActionResult  
AddProduct(Product  
    product)  
    {  
        if  
        (ModelState.IsValid)  
        {
```

```
db.Products.Add(product);
```

```
db.SaveChanges();
```

```
_____;
```

```
return
```



flexiquiz.com



```

        _____;

        return
RedirectToAction("Index");
    }

    return View(product);
}

public IActionResult
Index()
{
    if
(TempData.ContainsKey("SuccessMessage"))
    {

ViewBag.SuccessMessage
= _____;
    }

    return View();
}

```



flexiquiz.com



ViewBag.SuccessMessage =
"Produsul a fost adăugat cu
succes!";

☐ ViewBag.SuccessMessage

TempData["Message"] =
"Produsul a fost adăugat cu
succes!";

☐ TempData["Message"]

ViewData["SuccessMessage"] =
"Produsul a fost adăugat cu
succes!";

☐ ViewData["SuccessMessage"]

TempData["SuccessMessage"]
= "Produsul a fost adăugat cu
succes!";

☐ TempData["SuccessMessage"]

SUBMIT